c-for-emb

Автор Имя Фамилия

Содержание

1. Препроцессор и макрокоманды . . . 1
1.1. Директивы препроцессора . . . 1
1.2. Объектные макросы (макросы без аргументов) . . . 1
1.3. Функциональные макросы (макросы с аргументами) . . . 2
1.4. Операторы # и ## . . . 3
1.5. Предопределённые макросы . . . 3
1.6. Условная компиляция . . . 4
2. Многострочные макросы . . . 8
3. Достоинства и недостатки макросов . . . 9
3.1. Достоинства . . . 9
3.2. Недостатки . . . 9
4. Когда следует применять макросы . . . 10
4.1. Когда макросы оправданы . . . 10
4.2. Когда следует отказаться от макросов . . . 10
5. Резюме . . . 12
6. Типы в языке C . . . 13
6.1. Понятие типа . . . 13
6.2. Тип, объект, значение . . . 14
6.3. Базовые и производные типы . . . 15
6.4. Квалифицированные типы . . . 16
6.5. Общая картина типов в C . . . 19

Chapter 1. Препроцессор и макрокоманды

Мы уже очень бегло познакомились с ролью препроцессора в главе, посвященной компиляции (TBD). Пришла пора познакомиться с языком макросов детально, поскольку они являются весьма важной частью языка C в целом.

Прежде всего необходимо понять, что препроцессор — это отдельная программа, выполняющаяся **до** передачи исходного текста компилятору. Его задача — выполнить некоторые предварительные преобразования текста программы: подключить заголовочные файлы, раскрыть макроопределения, удалить комментарии и обработать директивы условной компиляции.



Препроцессор **не анализирует** синтаксис языка и **не выполняет** проверку типов. Он работает исключительно с текстом, производя в нём механические замены. Это свойство определяет основные опасности макрокоманд.

1.1. Директивы препроцессора

Все директивы препроцессора начинаются со знака `#`, который должен располагаться в начале строки (допустимы только пробельные символы перед ним). Директива **не требует** точки с запятой в конце и действует до конца строки.

Основные директивы:

Директива	Назначение
`#define`	Определение макроса
`#undef`	Отмена определения макроса
`#include`	Включение содержимого файла
`#ifdef`, `#ifndef`, `#if`, `#elif`, `#else`, `#endif`	Условная компиляция
`#error`, `#warning`	Выдача сообщения на этапе препроцессинга
`#pragma`	Передача указаний компилятору

1.2. Объектные макросы (макросы без аргументов)

Объектный макрос — это именованная текстовая последовательность, которая подставляется вместо имени макроса везде, где оно встречается в исходном тексте после точки определения.

Пример:

```
#define MAX_STUDENTS 30
#define PI 3.14159265358979
```

```
int group[MAX_STUDENTS];
double area = PI * r * r;
```

После работы препроцессора текст примет вид:

```
int group[30];
double area = 3.14159265358979 * r * r;
```

Объектные макросы традиционно используются для задания именованных констант. Это устраняет так называемые **магические числа** — безымянные числовые литералы, смысл которых неочевиден из контекста.

Макрос может быть определён и без тела — такое определение используется как **флаг** для **директив условной компиляции** (см. ниже):

```
#define DEBUG
```

Отменить определение макроса позволяет директива `#undef`:

```
#undef DEBUG
```

1.3. Функциональные макросы (макросы с аргументами)

Функциональный макрос принимает аргументы и подставляет их в тело определения. Синтаксически такой макрос напоминает вызов функции, однако механизм его работы принципиально иной: происходит не вычисление, а **текстовая подстановка**.

```
#define SQUARE(x) ((x) * (x))
#define MAX(a, b) ((a) > (b) ? (a) : (b))

int y = SQUARE(5);       /* y = ((5) * (5)) */
int m = MAX(a + 1, b);   /* m = ((a + 1) > (b) ? (a + 1) : (b)) */
```

Обратите внимание на скобки вокруг каждого аргумента и вокруг всего тела макроса. Это не излишество, а необходимость, связанная с приоритетом операций. Посмотрим, что будет без таких скобок:

```
#define BAD_SUM(a, b) a + b
int x = BAD_SUM(2, 3) * 4;
```

Раскрытие даст `2 + 3 * 4`, то есть `14`, а не ожидаемые `20`. Правильное определение:

```
#define GOOD_SUM(a, b) ((a) + (b))
```

Его раскрытие даст `2) + (3 * 4`, в соответствии с ожидаемым результатом.

1.4. Операторы # и

В теле функционального макроса могут использоваться два специальных оператора.

1.4.1. Оператор # ("стрингификация")

Оператор `#`, стоящий перед параметром макроса, превращает фактический аргумент в строковый литерал.

```
#define PRINT_VAR(var, fmt) \
    printf(#var " = %" #fmt "\n", var)

int count = 42;
PRINT_VAR(count, d);
/* Раскрывается в:
   printf("count" " = %" "d" "\n", count);
   что эквивалентно:
   printf("count = %d\n", count);
*/
```

1.4.2. Оператор ## (конкатенация)

Оператор `##` склеивает два лексических элемента в единый идентификатор.

```
#define MAKE_FUNC(name) void func_##name(void)

MAKE_FUNC(init);   /* Раскрывается в: void func_init(void) */
MAKE_FUNC(cleanup); /* Раскрывается в: void func_cleanup(void) */
```

Эти операторы позволяют писать макросы, генерирующие имена идентификаторов и строк, что невозможно реализовать средствами самого языка C.

1.5. Предопределённые макросы

Компилятор предоставляет ряд макросов, значения которых определяются автоматически. Наиболее употребительные из них:

Макрос	Значение
`__LINE__`	Номер текущей строки исходного файла
`__FILE__`	Имя текущего исходного файла (строковый литерал)

Макрос	Значение
`__DATE__`	Дата компиляции в формате "Mmm dd yyyy"
`__TIME__`	Время компиляции в формате "hh:mm:ss"
`__STDC__`	Определён, если компилятор соответствует стандарту (C)
`__STDC_VERSION__`	Номер версии стандарта C (например, `201112L` для C11)

Эти макросы широко применяются в отладочных сообщениях:

```
#define LOG_ERROR(msg) \
    fprintf(stderr, "Ошибка в %s:%d — %s\n", __FILE__, __LINE__, msg)
```

1.6. Условная компиляция

Директивы условной компиляции позволяют включать или исключать фрагменты текста программы в зависимости от выполнения некоторых условий. Это необходимо, в частности, для:

- адаптации программы к различным операционным системам и архитектурам;
- включения отладочных фрагментов только в специальных сборках;
- предотвращения повторного включения заголовочных файлов.

1.6.1. Директива `#if`

Директива `#if` позволяет включать или исключать блоки кода из компиляции в зависимости от значения константного выражения.

Сначала директива `#if` вычисляет целочисленное константное выражение, составленное из литералов и макросов. Если указанное выражение после `#if` вычисляется как ненулевое (то есть истинно), то весь код, который находится между `#if` и соответствующей закрывающей директивой `#endif`, включается в итоговый исходный файл для компилятора. Если же выражение равно нулю (ложно), то этот код полностью игнорируется и в компиляцию не попадает.

Синтаксис:

```
#if выражение
    // код для компиляции, если выражение истинно
#endif
```



Условие должно быть константным выражением (то есть его значение вычисляется на этапе компиляции).



Директива `#if` всегда парная. Для каждой такой директивы должна быть

> соответствующая закрывающая `#endif`. Если этого не сделать, компилятор выдаст ошибку.

Внутри блока `#if` можно использовать директивы `#elif` (аналог `else if`), `#else` (альтернативный блок для случая, когда `#if` ложен) и вложенные `#if`. При этом `#else`, если он есть, должен быть последней директивой перед `#endif`.

Пример использования:

```
#define DEBUG 1

#if DEBUG
    printf("Отладка включена\n");
    // какие-то отладочные функции
#endif

// Нормальная работа
```

В этом примере, если макрос `DEBUG` определён и не равен нулю, в итоговый файл попадут вызов `printf` и `какие-то отладочные функции`; в противном случае они будут исключены из компиляции.

```
#define VERSION 2

#if VERSION >= 2
    void new_feature(void);
#else
    void old_feature(void);
#endif
```

В этом примере производится проверка номера версии. В случае, если номер версии 2 или выше, вызывается функция `new_feature`, в противном случае — `old_feature`.

1.6.2. Директивы `#ifdef` и `#ifndef`

Директива `#ifdef NAME` эквивалентна `#if defined(NAME)`, а `#ifndef NAME` — `#if !defined(NAME)`. Оператор `defined(NAME)` возвращает истинное значение, если макроопределение `#define NAME` было встречено ранее (возможно, без указания значения), и ложное — в противном случае.

```
#ifdef DEBUG
    printf("Отладочное сообщение\n");
#endif

#ifndef MAX_BUFFER
    #define MAX_BUFFER 1024
#endif
```

Оператор `defined` особенно удобен в сложных условиях:

```
#if defined(OS_LINUX) && !defined(DISABLE_NETWORKING)
    /* код для Linux с поддержкой сети */
#elif defined(OS_WINDOWS)
    /* код для Windows */
#else
    #error "Неподдерживаемая операционная система"
#endif
```

1.6.3. Защита от повторного включения

Директивы условной компиляции часто используются в идиоме, которая получила название **include guard**. Она предназначена для решения проблемы повторного включения заголовочного файла.

Эта проблема заключается в следующем. Предположим, в нашем проекте используются заголовочные файлы `a.h` и `b.h`. Каждый из них, в свою очередь, включает вложенный заголовочный файл `c.h` через директиву `#include` (многоуровневое вложение одних заголовочных файлов в другие — обычная практика.

Допустим, некоторый модуль `d.c` включает и `a.h`, и `b.h`:

```
#include "a.h" ①
#include "b.h" ②
```

Сначала препроцессор выполнит первую директиву `#include` (1). Он включит в исходный код модуля `d.c` содержимое заголовочного файла `a.h`, включая вложенный файл `c.h`. Затем будет выполнена вторая директива `#include` (2), в ходе которой в модуль `d.c` будет включено содержимое `b.h`, в том числе (повторно!) и `c.h`.

Когда компилятор дойдет до второй копии `c.h`, он встретит повторные определения типов, переменных и макросов, которые он уже обработал ранее. Это приведет к ошибке компиляции (повторное определение).

Эту проблему нельзя решить, просто удалив директиву `#include "c.h"` из файла `b.h`, поскольку в проекте могут быть и другие модули, которые включают только `b.h`. Теперь ошибка возникнет уже при их компиляции, поскольку необходимые определения из `c.h` теперь недоступны.

На помощь приходит **include guard**:

```
#ifndef MYHEADER_H ①
#define MYHEADER_H ②

/* содержимое заголовочного файла */
```

```
#endif /* MYHEADER_H */ ③
```

При первом включении заголовочного файла макрос `MYHEADER_H` не определен, поэтому условие (1) истинно, и выполняется остальной текст макроопределения вплоть до `#endif` (3). Будет опроеделен макрос `MYHEADER_H` (2), а затем обработано содержимое заголовочного файла.

При втором и последующих включениях макрос `MYHEADER_H` уже определен, поэтому условие (2) ложно, и содержимое заголовочного файла игнорируется.

Альтернативой является нестандартная, но широко поддерживаемая директива:

```
#pragma once
```

При ее использовании всегда есть (небольшой) риск, что в данной версии компилятора она не реализована.

Chapter 2. Многострочные макросы

Длинное определение макроса можно разбить на несколько строк, экранируя перевод строки обратным слешем:

```
#define INIT_STRUCT(s) do { \
    (s).x = 0;              \
    (s).y = 0;              \
    (s).flags = 0;          \
} while (0)
```



Обратите внимание на конструкцию `do { … } while (0)`. Она необходима, чтобы макрос, раскрывающийся в несколько операторов, синтаксически вёл себя как **единый оператор**. Это позволяет безопасно использовать его в конструкциях `if-else`:

```
if (condition)
    INIT_STRUCT(obj);   /* без do-while здесь возникла бы ошибка */
else
    /* ... */
```

Компилятор распознаёт, что цикл выполняется ровно один раз, и оптимизирует его, не генерируя лишнего машинного кода.

Chapter 3. Достоинства и недостатки макросов

Макрокоманды — мощный, но специфический механизм. Они стали неотъемлемой частью программирования на C и оказываются очень полезными при правильном применении. Однако использовать их нужно с большой осторожностью и всегда отдавать себе отчет, чтьо вы делаете.

3.1. Достоинства

1. **Отсутствие накладных расходов.** Макрос раскрывается на этапе препроцессинга, поэтому вызов макроса не связан с передачей управления, формированием стекового кадра и возвратом из функции. Это важно в системах с ограниченными ресурсами.
2. **Работа с синтаксисом языка.** Макросы способны генерировать идентификаторы (через `#), преобразовывать их в строки (через`) и использовать информацию о месте вызова (`FILE`, `LINE`). Ни одна языковая конструкция C на это не способна.
3. **Условная компиляция.** Препроцессор позволяет включать и исключать фрагменты программы в зависимости от конфигурации сборки.
4. **Генерация шаблонного кода.** С помощью макросов можно создавать семейства однотипных функций и структур, что востребовано в системном программировании.

3.2. Недостатки

1. **Отсутствие проверки типов.** Препроцессор не различает типы данных. Ошибка, связанная с несовместимостью аргументов, проявится лишь на этапе компиляции раскрытого текста, что затрудняет локализацию проблемы.
2. **Многократное вычисление аргументов.** Если аргумент макроса используется в теле несколько раз, он вычисляется соответствующее число раз. Это порой может привести к неожиданному результату:

   ```
   #define SQUARE(X) ((X) * (X))
   ```

   Вызов `SQUARE(x++)` приведёт к двойному инкременту переменной `x`, что является источником трудноуловимых ошибок.

3. **Сложность отладки.** Отладчик работает с раскрытым текстом, а не с исходным макросом. Сообщения об ошибках указывают на строки раскрытия, а не на строки исходного кода.
4. **Нарушение области видимости.** Макрос действует глобально от точки определения до конца файла или до директивы `#undef`. Он не подчиняется правилам блочной области видимости языка C.

Chapter 4. Когда следует применять макросы

Общее правило: **если задачу можно легко решить средствами самого языка C, макрос использовать не следует**.

4.1. Когда макросы оправданы

- **Условная компиляция** — для адаптации программы к различным платформам и конфигурациям сборки. Альтернатив здесь нет.
- **Защита заголовочных файлов** от повторного включения.
- **Отладочные и логгирующие макросы**, использующие `FILE` и `LINE`.
- **Генерация однотипного кода** — например, семейство функций для разных типов данных, когда обобщённые средства языка недостаточны.
- **Короткие выражения**, где важна каждая операция (в критичных по производительности участках встраиваемых систем).

4.2. Когда следует отказаться от макросов

Задача	Плохо (макрос)	Хорошо (средства C)
Именованные целочисленные константы	`#define SIZE 100`	`enum { SIZE = 100 };`
Простые функции	`#define SQUARE(x) x)*(x`	`static inline int square(int x) { return x * x; }`
Полиморфные операции	`#define MAX(a,b) …`	Отдельные функции `max_i`, `max_d` или `_Generic` (C11)
Строковые константы	`#define MSG "text"`	`static const char MSG[] = "text";`

Конструкция `static inline` (стандарт C99) позволяет получить функцию, тело которой встраивается в место вызова, то есть по сути достичь того же отсутствия накладных расходов на вызов функции, что и у макроса, но с полной проверкой типов и корректной работой отладчика.

Оператор `_Generic` (стандарт C11) даёт возможность выбирать реализацию в зависимости от типа аргумента, что позволяет создавать типобезопасные полиморфные конструкции без текстовых подстановок.

```
static inline int    max_i(int a, int b)       { return a > b ? a : b; }
static inline double max_d(double a, double b) { return a > b ? a : b; }

#define MAX(a, b) _Generic((a), \
    int:    max_i,              \
```

```
    double: max_d              \
)(a, b)
```

В этом определении макрос используется лишь как тонкая оболочка над типобезопасными функциями, что существенно снижает связанные с макросами риски.

Chapter 5. Резюме

Макрокоманды — неотъемлемая часть языка C. Они предоставляют возможности, недоступные языковым средствам: условную компиляцию, работу с именами идентификаторов, генерацию кода. Однако их простая текстовая природа делает макросы источником труднодиагностируемых ошибок.

Современный стиль программирования на C предполагает **минимизацию** использования макросов. Там, где раньше применялись макросы для констант и простых функций, сегодня следует использовать `enum` и `static inline`. Макросы должны оставаться там, где они действительно незаменимы: в условной компиляции, в отладочных конструкциях и в задачах генерации кода.

Chapter 6. Типы в языке C

В этой главе мы рассмотрим типы объектов в языке C. Но сначала уточним, что следует понимать под типом. Это важно, поскольку понятие типа часто трактуется неверно.

В качестве эксперимента я задавал этот вопрос нескольким нейронным сетям и периодически получал некорректный ответ, как от новичка в языке C на собеседовании. Это значит, что велика вероятность войти в заблуждение при попытке прояснить этот вопрос самостоятельно при помощи сетевых ресурсов.

6.1. Понятие типа

Тип объекта определяется двумя фундаментальными характеристиками:

Домен значений

Область значений, которые объект данного типа способен представлять. Например:

- для типа `int` это целые числа;
- для типа `double` — числа с плавающей десятичной точкой, представленные в формате "мантисса + порядок";
- для типа `bool` — одно из двух значений, `false` либо `true`;
- для типа `struct { int x; int y; }` — все возможные пары целых чисел.

Множество операций

Совокупность операций, определённых над значениями данного типа, а также их семантика. Например:

- оператор `/` для целых типов выполняет деление **нацело** с отбрасыванием дробной части, для вещественных — обычное арифметическое деление;
- оператор `++` применительно к целому увеличивает значение на единицу, а применительно к указателю — смещает его на размер элемента.



Вопреки распространенному заблуждению, тип сам по себе не определяет размер объекта в памяти. Например, тип `int` имеет различный размер для разных архитектур (типичный размер 16 либо 32 бита); одна и та же структура может иметь различный размер даже на одной архитектуре , если используются разые варианты выравнивания ее компонентов. При этом домен и множество операций над этими типами остаются принципиально теми же. Бесполезно использовать предыдущий опыт в этом вопросе, поскольку простое изменение опций компиляции или атрибутов могут существенно изменить представление объекта в памяти, не говоря уже о переходе на другую платформу (вы же помните, одно из главных преимуществ языка C — это возможность писать переносимый код, одинаково работающий на разных платформах).

Примеры:

На многих платформах типы `int` и `float` имеют одинаковый размер в памяти (по 32 бита), однако они остаются совершенно разными типами, поскольку имеют разные домены значений, разные наборы операций и даже различную семантику одинаковых операций (см. пример с делением).

Одина и та же битовая последовательность может интерпретироваться по-разному. Например, 4 байта `0x48 0x65 0x61 0x64` может быть интерпретировано как:

1. целое число `0x64616548 = 1,684,104,520` (на машине с порядком байтов little-endian);
2. адрес ячейки оперативной памяти (указатель);
3. представление числа 1.6631×10^{22} в формате IEEE 754 single precision (32 бита);
4. текстовая строка "Head" (в кодировке Windows-1251 ;
5. часть объекта агрегатного типа (массива или структуры).

Выбор интерпретации бинарного представления полностью определяется типом объекта.

Пример разной семантики оператора деления в зависимости от типа операндов:

```
int   i = 1;
float f = 1.0f;

i / 2;    /* 0    – домен целых, деление с отбрасыванием */
f / 2;    /* 0.5f – домен вещественных, обычное деление */
```



При анализе типа целесообразно задавать три вопроса:

1. Значения из какого домена он способен хранить?
2. Каковы ограничения домена (что происходит при переполнении, когда значение выходит за пределы допустимого диапазона)?
3. Какие операции над типом определены стандартом и какова их семантика?

6.2. Тип, объект, значение

Следует разграничивать три связанных понятия:

- **Тип** — характеристика, определяющая домен значений и допустимые операции.
- **Объект** — область памяти, способная хранить значение определённого типа (т. е. экземпляр типа).
- **Значение** — конкретный элемент домена, хранящийся в объекте.



Функция обладает **типом**, но не является **объектом**.



Тип `void` существует, однако объект этого типа создать нельзя.

Пример:

```
int i = 42;
```

В данном примере объект `i` имеет целочисленный тип (`int`) и инициализируется значением этого типа (`42`).

6.3. Базовые и производные типы

Типы в языке C можно разделить на две большие категории: **базовые** (fundamental types) и **производные** (derived types). Базовые типы являются примитивными — они задаются самим языком и не строятся из других типов. Производные типы конструируются из уже существующих (базовых или других производных) по определённым правилам.

6.3.1. Базовые типы

К базовым типам относятся:

- `void` — тип без значения. Используется для обозначения отсутствия результата функции, отсутствия параметров, а также как базовый тип для указателя `void *` («указатель на объект неизвестного типа»).
- **Арифметические типы**, которые, в свою очередь, делятся на:
  - **целочисленные** (integer types):
    - символьные: `char`, `signed char`, `unsigned char`;
    - обычные целые: `short`, `int`, `long`, `long long` (каждый в знаковой и беззнаковой форме);
    - логический: `_Bool` (в C23 — также псевдоним `bool`);
    - расширенные символьные (C23): `char8_t`, `char16_t`, `char32_t`.
  - **типы с плавающей точкой** (floating types): `float`, `double`, `long double`;
  - **десятичные плавающие** (C23): `_Decimal32`, `_Decimal64`, `_Decimal128`;
  - **комплексные** (C99): `_Complex`-варианты для каждого плавающего типа.
- **Перечисления** (enumerated types, `enum`) — по стандарту относятся к целочисленным типам, но объявляются пользователем и поэтому часто рассматриваются отдельно.

```
int i;              /* базовый целочисленный тип */
double x;           /* базовый плавающий тип */
void f(void);       /* void как тип результата и тип параметров */
```

6.3.2. Производные типы (derived types)

Производные типы конструируются из других типов рекурсивно. К ним относятся:

- **указатели** — `T *` (указатель на объект типа `T`);
- **массивы** — `T[N]` (массив из `N` элементов типа `T`);
- **структуры** — `struct { ⋯ }` (агрегат полей различных типов);
- **объединения** — `union { ⋯ }` (поля, разделяющие одну область памяти);
- **функции** — `T(params)` (функция, возвращающая значение типа `T`);

Производные типы могут строиться из других производных типов, что позволяет описывать сколь угодно сложные структуры данных:

```
int *p;                    /* указатель на int */
int **pp;                  /* указатель на указатель на int */
int a[10];                 /* массив из 10 int */
int *ap[5];                /* массив из 5 указателей на int */
int (*pa)[5];              /* указатель на массив из 5 int */
int (*f)(double);          /* указатель на функцию (double) -> int */
int (*af[3])(double);      /* массив из 3 указателей на функцию */
```

Обратите внимание, что тип переменной в случае производных типов определяется не одним ключевым словом, а всей конструкцией объявления: в `int (*pa)[5]` типом переменной `pa` является «указатель на массив из 5 элементов типа `int`» — производный тип, построенный рекурсивно из базового `int`.

6.4. Квалифицированные типы

Любой тип (базовый или производный) может быть снабжён одним или несколькими квалификаторами:

- `const` — объект не может быть изменён через данное имя;
- `volatile` — объект может изменяться способами, невидимыми компилятору (например, аппаратурой);
- `restrict` (только для указателей) — указатель является единственным способом доступа к объекту;
- `_Atomic` (C11) — доступ к объекту производится атомарно.

```
const int *p;              /* указатель на const int */
volatile int * const p;    /* const-указатель на volatile int */
```

Такие типы называются **квалифицированными**.

Квалифицированный тип считается **отличным** от неквалифицированного, но **совместимым** с ним. В этом заключается важное различие между **идентичностью** типа и

совместимостью типов.

6.4.1. Что значит «отличный» (distinct)?

Когда мы говорим, что `const int` **отличен** от `int`, мы имеем в виду, что это **разные типы** с точки зрения системы типов языка. Они **не** являются одним и тем же типом.

```
/* Эти два объявления имеют РАЗНЫЕ типы */
int x;
const int y;

/* sizeof(int) и sizeof(const int) дают одинаковый результат,
   но типы всё равно считаются различными */
```

6.4.2. Что значит «совместимый» (compatible)?

Несмотря на то, что типы различны, они **совместимы** — это означает, что в большинстве контекстов они могут использоваться взаимозаменяемо. Совместимость — это более слабое отношение, чем идентичность.

Примеры совместимости:

```
int x = 5;
const int y = x;        /* OK: int -> const int */
int z = y;              /* OK: const int -> int (копирование значения) */

void f(int a);
const int c = 10;
f(c);                   /* OK: передача const int в параметр типа int */
```

6.4.3. Как проявляется это различие?

Различие между «отличным» и «совместимым» проявляется в нескольких ситуациях.

Присваивание указателей

```
int x = 5;
const int *p1 = &x;     /* OK: int* -> const int* (добавление ограничения const) */

const int y = 10;
int *p2 = &y;           /* ОШИБКА: const int* -> int* (снятие ограничения const
запрещено) */
```

Здесь `int` и `const int *` — `*различные` типы, но они **совместимы** только в одну сторону: можно добавить ограничение `const`, но нельзя его снять.

Совместимость типов в объявлениях

```
/* Эти два объявления совместимы */
extern int x;
extern const int x;     /* OK: типы совместимы */

/* А это разные типы! */
int a[10];
const int b[10];        /* Это ДРУГОЙ тип массива */
```

Передача параметров и возврат значений

```
int f(void);
const int g(void);      /* OK: типы совместимы, но g возвращает const int */

void h(int x);
void h(const int x);    /* ОШИБКА: повторное объявление с другим типом параметра */
```

6.4.4. Практические последствия

«Отличный, но совместимый» означает:

- типы **не идентичны** (это разные типы в системе типов);
- но они **могут использоваться вместе** в большинстве контекстов благодаря правилам неявного преобразования;
- однако в некоторых строгих контекстах (например, повторные объявления функций, совместимость указателей) различие становится важным.

Это различие позволяет языку быть гибким (можно передавать `const int` туда, где ожидается `int`), но при этом сохранять строгость типизации (нельзя модифицировать const-объект через указатель на не-const).

Когда проявляется «отличность»

```
/* Функции с разными типами параметров */
void f(int x);
void f(const int x);    /* ОШИБКА: конфликтующие объявления */

/* Указатели на функции */
void (*p1)(int);
void (*p2)(const int);  /* Это РАЗНЫЕ типы указателей на функции */
```

Когда проявляется «совместимость»

```
/* Присваивание значений */
int a = 5;
const int b = a;        /* OK: значения совместимы */

/* Передача в функцию */
void process(int value);
const int c = 10;
process(c);             /* OK: const int совместим с int при передаче по значению */
```

6.4.5. Типы объектов и типы функций

Типы объектов — это типы, которые могут описывать объекты в памяти и, следовательно, имеют размер (`sizeof`). Все базовые типы (кроме `void`), указатели, массивы, структуры, объединения и атомарные типы являются типами объектов.

Функции — единственный вид производных типов, не являющийся типом объекта.

6.4.6. Завершённые (complete) и незавершённые (incomplete) типы

Тип считается **незавершённым**, если его размер невозможно определить в текущей точке программы. К незавершённым типам относятся `void`, массивы без указания размера (`int a[]`), структуры и объединения без определения тела.

Незавершённый тип становится **завершённым**, когда появляется недостающая информация (например, после определения структуры).

```
struct S;                   /* S — незавершённый тип */
int a[];                    /* a — массив незавершённого типа */
extern int b[];             /* допустимо для внешних объявлений */

struct S { int x; };        /* теперь S — завершённый тип */
```

Использование незавершённого типа там, где требуется знание его размера (например, в `sizeof` или при объявлении объекта), является ошибкой.

6.5. Общая картина типов в C

Все типы языка можно разделить на несколько категорий:

Категория	Примеры	Домен значений
Скалярные	`int`, `double`, `bool`, `char *`	Одно значение
Массивы	`int[10]`, `double[3][4]`	Упорядоченный кортеж из N значений
Структуры	`struct Point`	Произведение доменов полей

Категория	Примеры	Домен значений
Объединения	`union Value`	Ровно одно из представлений полей
Функции	`int (const char *)`	Сигнатура[1]
`void`	`void`, `void *`	Пустой домен

[1] Понятие сигнатуры функции зависит от языка и, в частности, от возможности перегрузки функций в этом языке. В C перегрузка функций остутствует, поэтому принято считать, что возвращаемое функцией значение входит в состав ее сигнатуры наряду с ее именем и типами параметров. В других языках, например, C++, это не так.